

Security Analysis – Drone Management System

2026-03-25

Overview

The codebase applies several solid security foundations: parameterized SQL queries throughout, bcrypt (cost 12) with HMAC pepper for password hashing, session-bound CSRF tokens compared with `crypto.timingSafeEqual`, HTTP-only / SameSite session cookies with forced regeneration at login, and role-based access control across three privilege tiers. The following sections document the gaps that remain.

Critical Issues

1. Drone token rotation stores raw token (broken hashing)

`src/models/drone_management.js` — `create_drone_with_defaults` correctly hashes the drone token with `hmac_sha256_hex` before insertion (line 35). However, `rotate_drone_token` (line 101–108) passes the raw, unhashed token directly to the UPDATE query. After any rotation the stored value is plaintext, making all rotated tokens immediately usable if the database is compromised. The fix is to apply the same `hmac_sha256_hex` wrapper before the UPDATE.

2. resend-invite references `target_id` before declaration (ReferenceError)

`src/routes/admin_users.js`, line 227 passes `target_id` inside the `audit()` call that fires when `is_super_admin` is false — but `target_id` is not declared until line 231 (`const target_id = parse_user_id(...)`). Because `const` has a temporal dead zone in strict mode ('use strict' is set), this branch always throws a `ReferenceError` instead of returning 403. Any non-super-admin who attempts to resend an invite triggers an unhandled exception rather than a clean denial.

3. Audit logs are console-only and lost on restart `src/utils/audit_logger.js` emits every admin action as `console.log`. If the process restarts, is replaced, or its stdout is not captured by a log aggregator, the entire audit trail is silently discarded. For a system that controls physical drones this is a compliance and forensics gap. Audit records should be written to a dedicated database table (append-only, non-deletable by the application user) as the primary store, with console output as secondary.

High-Priority Issues

4. Content Security Policy is explicitly disabled `src/app.js` line 22:

`contentSecurityPolicy: false`. Helmet is loaded but the most impactful header is turned off. A CSP with `default-src 'self'`, `script-src 'self'`, and `frame-ancestors 'none'` should be enabled. This is the single cheapest mitigation against reflected or stored XSS.

5. Active sessions survive user suspension

When an admin calls `POST /admin/users/:id/status` to suspend an account, existing sessions for that user remain valid until the next request triggers `refresh_session_user`. There is a window — potentially hours given the default 7-day `maxAge` — in which a suspended account retains full access. Sessions should be deleted from the `sessions` table immediately on status change.

6. Rate limiting is IP-only and relatively permissive `src/security/rate_limit.js`

allows 10 login attempts per 10 minutes per IP and 8 password-reset requests per hour per IP. Both limits can be bypassed trivially by rotating source IPs or using a proxy. Recommended additions: per-account failed-attempt counter with exponential back-off or temporary lockout, and a stricter window (e.g. 5 attempts / 15 minutes) aligned with NIST SP 800-63B guidance.

7. Password reset links are logged in development mode `src/routes/auth.js` line 165–169 and `src/routes/admin_users.js` line 74–76 write complete reset URLs (containing live tokens) to `console.log` when `LOG_PASSWORD_RESET_LINKS=true` (the default outside production). If development logs are forwarded to a shared log aggregator or accidentally committed, tokens are exposed. Logging should reference only a token ID, never the full token value.

Medium-Priority Issues

8. Email validation accepts any string containing @ `src/utils/validators.js` line 19–25 only checks for @ presence and string length. Values like `x@` or `@@` pass. A simple RFC 5322-compatible regex or a library such as `validator.js` (`isEmail`) should be used. This also reduces the risk of header injection in the `nodemailer` calls.

9. Database connections have no explicit SSL/TLS configuration `src/db/pool.js` and `src/auth/session.js` (MySQL session store) create connections without an `ssl` option. If the database host is separate from the application host, credentials and session data transit in plaintext. An `ssl: { rejectUnauthorized: true }` option referencing the CA certificate should be added for any non-localhost deployment.

10. `trust proxy: 1` — IP spoofing risk for rate limiting `src/app.js` line 19 trusts exactly one upstream proxy for `X-Forwarded-For`. If the deployment topology places the

application behind more than one reverse proxy (load balancer + edge proxy), the wrong IP is used for rate limiting, allowing an attacker to spoof the source address. The value should match the actual infrastructure (a specific IP, a CIDR range, or a higher hop count).

Best-Practice Recommendations

#	Finding	Severity	Effort
1	Drone token rotation stores raw token	Critical	Low
2	resend-invite ReferenceError (TDZ)	Critical	Low
3	Audit logs not persisted	Critical	Medium
4	Content Security Policy disabled	High	Low
5	No session invalidation on suspension	High	Medium
6	IP-only rate limiting	High	Medium
7	Reset links logged in dev	High	Low
8	Weak email validation	Medium	Low
9	No database SSL configuration	Medium	Medium
10	Trust proxy misconfigured	Medium	Low

Additional longer-term improvements: introduce TOTP/WebAuthn MFA for admin and super-admin accounts; add password complexity rules beyond minimum length; implement session lifecycle logging (creation, destruction, suspicious activity); and replace the placeholder `action_placeholder` responses in `src/routes/user_drones.js` with properly authorized and audited drone action handlers before the system reaches production.